



An empirical study of various candidate selection and partitioning techniques in the DIRECT framework

Linās Stripinis¹ · Remigijus Paulavičius¹

Received: 19 September 2021 / Accepted: 12 May 2022

© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2022

Abstract

Over the last three decades, many attempts have been made to improve the DIRECT (DIviding RECTangles) algorithm's efficiency. Various novel ideas and extensions have been suggested. The main two steps of DIRECT-type algorithms are selecting and partitioning potentially optimal rectangles. However, the most efficient combination of these two steps is an area that has not been investigated so far. This paper presents a study covering an extensive examination of various candidate selection and partitioning techniques within the same DIRECT algorithmic framework. Twelve DIRECT-type algorithmic variations are compared on 800 randomly generated GKLS-type test problems and 96 box-constrained global optimization problems from DIRECTGOLib v1.1 with varying complexity. Based on these studies, we have identified the most efficient selection and partitioning combinations leading to new, more efficient, DIRECT-type algorithms. All these algorithms are included in the latest version of DIRECTGO v1.1.0 and are publicly available.

Keywords DIRECT-type algorithm · Global optimization · Derivative-free optimization

Mathematics Subject Classification 90C26 · 90C56

1 Introduction

In this paper, we consider a box-constrained global optimization problem of the form:

$$\min_{\mathbf{x} \in D} f(\mathbf{x}) \quad (1)$$

where $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is a Lipschitz-continuous, potentially “black-box” objective function, and $\mathbf{x}$ is the input vector. Thus, we assume that the analytical information of the objective function f is unknown and can only be obtained by evaluating f at various points of the

✉ Remigijus Paulavičius
remigijus.paulavicius@mif.vu.lt

Linās Stripinis
linas.stripinis@mif.vu.lt

¹ Vilnius University, Institute of Data Science and Digital Technologies, Akademijos 4, LT-08663 Vilnius, Lithuania

feasible region, which is an n -dimensional hyper-rectangle

$$D = [\mathbf{a}, \mathbf{b}] = \{\mathbf{x} \in \mathbb{R}^n : a_j \leq x_j \leq b_j, j = 1, \dots, n\}.$$

Moreover, f can be non-linear, multi-modal, non-convex, and non-differentiable.

The optimization community attracted considerable interest from the simplicity and efficiency of the deterministic DIRECT-type algorithms. The original DIRECT algorithm was developed by Jones et al. [25] and is a well-known and widely used solution technique for derivative-free global optimization. The DIRECT algorithm extends classical Lipschitz optimization [36–39, 42, 43, 47, 49], where the need for the Lipschitz constant is eliminated. This feature made DIRECT-type methods especially attractive for solving various real-world optimization problems (see, e.g., [1–3, 5, 6, 11, 31, 35, 41, 53] and the references given therein). Furthermore, the extensive numerical benchmarks in [44] revealed an encouraging performance of the DIRECT algorithm among other tested derivative-free global optimization approaches, belonging to genetic [22], simulated annealing [27], and particle swarm optimization [26].

Typically, the DIRECT-type algorithms include three main steps: selection, sampling, and partitioning (subdivision). At each iteration, a specific DIRECT-type algorithm identifies (selects) the set of potentially optimal hyper-rectangles (POHs) and then samples and subdivides them. The original DIRECT algorithm uses hyper-rectangular subdivisions based on n -dimensional trisection. The objective function is evaluated at the center points of the newly-formed sub-rectangles. Moreover, if several dimensions have the maximum side length, DIRECT starts trisection from the dimension with the lowest w_j and continues to the highest [24, 25]. Here w_j is defined as the best function values sampled along dimension j

$$w_j = \min\{f(\mathbf{c} + \delta \mathbf{e}_j), f(\mathbf{c} - \delta \mathbf{e}_j)\}, \quad (2)$$

where $j \in M$ (set of dimensions with the maximum side length), δ is equal to one-third of the maximum side length, $\mathbf{c}$ is the center of the hyper-rectangle, and $\mathbf{e}_j$ is the j th unit vector. Figure 1 illustrates the selection, sampling, and subdivision (trisection) in the original DIRECT algorithm for a two-dimensional *Branin* test function.

Since the original DIRECT algorithm was published, various DIRECT-type extensions and modifications have been proposed. One large group of existing modifications aim to improve the selection of POHs (see, e.g., [1, 10, 32, 35, 52]), while the other group concentrates on different partitioning techniques (see, e.g., [23, 28, 33, 40, 45]). In addition, the authors also make some modifications to the other steps of their algorithms. Consequently, it is unclear which suggested improvements have the most potential within the DIRECT algorithmic framework.

We address this problem by comparing various proposed candidate selection and partitioning techniques for the remaining algorithmic steps under the same conditions. This way, we seek to improve the efficiency of existing DIRECT-type algorithms by creating new combinations based on the previous proposals. Twelve mostly new DIRECT-type algorithmic variations are introduced and investigated using three selection and four partitioning schemes.

The rest of the paper is organized as follows. Section 2 reviews the original DIRECT algorithm and well-known DIRECT-type modifications proposed for the candidate selection and subdivision. The obtained new combinations are described in Sect. 3. An extensive experimental analysis using traditional test problems is presented in Sect. 4, while on GKLS-type test problems in Sect. 5. Finally, in Sect. 6, we conclude the paper.

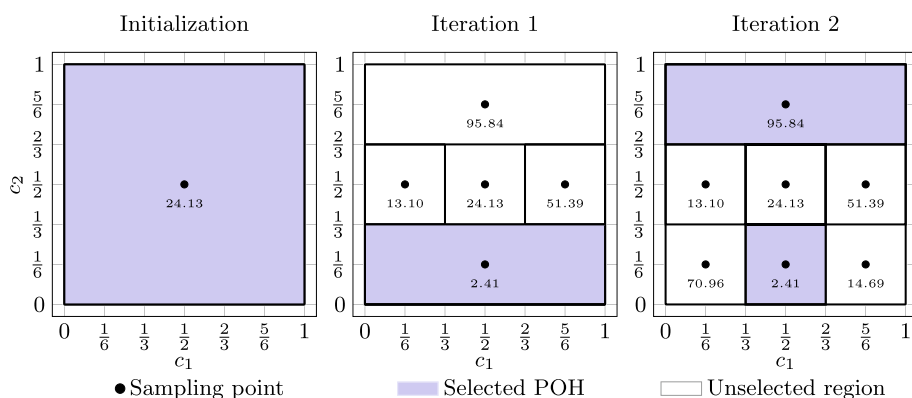


Fig. 1 Illustration of selection, sampling, and subdivision (trisection) used in the original DIRECT algorithm [25] on a two-dimensional *Branin* test function in the first two iterations

2 Overview of candidate selection and partitioning techniques used in DIRECT-type algorithms

This section reviews the most well-known strategies for selecting and partitioning potentially optimal candidates in DIRECT-type algorithms. We start with a brief review of the main steps of the original DIRECT algorithm, with particular emphasis on candidate selection and partitioning techniques.

2.1 Original DIRECT algorithm

The original DIRECT algorithm is a deterministic derivative-free global optimization [20, 48, 56] algorithm subject to simple box constraints. The main steps of DIRECT are summarized in Algorithm 1. At the **Initialization** step (see Algorithm 1, Lines 2–7), DIRECT normalizes the search region D to unit hyper-rectangle $\bar{D}$ and refers to the original space D only when evaluating the objective function. Regardless of the dimension n , the first evaluation of the objective function is performed at the midpoint of the unit hyper-rectangle $\mathbf{c}^1 = (1/2, \dots, 1/2) \in \bar{D}$.

Two of the most critical steps in the original DIRECT and other existing modifications are **Selection** and **Subdivision**.

2.1.1 Original selection strategy

Let the current partition at the iteration k is defined as

$$\mathcal{P}_k = \{\bar{D}_k^i : i \in \mathbb{I}_k\},$$

where $\bar{D}_k^i = [\mathbf{a}^i, \mathbf{b}^i] = \{\mathbf{x} \in \bar{D} : 0 \leq a_j^i \leq x_j \leq b_j^i \leq 1, j = 1, \dots, n, \forall i \in \mathbb{I}_k\}$ and $\mathbb{I}_k$ is the index set identifying the current partition $\mathcal{P}_k$. The next partition, $\mathcal{P}_{k+1}$, is obtained by subdividing selected POHs from the current partition $\mathcal{P}_k$. Note there is only one candidate, $\bar{D}_1^1$, that at the first iteration ($k = 1$), which is automatically potentially optimal. The formal requirement of potential optimality in subsequent iterations is stated in Definition 1.

Algorithm 1: Main steps of the DIRECT algorithm

```

1 DIRECT( $f, D, opt$ );
   input : Objective function ( $f$ ), search domain ( $D$ ), and adjustable algorithmic
           options ( $opt$ ): tolerance ( $\varepsilon_{pe}$ ), maximal number of function evaluations
           ( $M_{max}$ ) and algorithmic iterations ( $K_{max}$ );
   output: The best found objective value ( $f^{\min}$ ), solution point ( $\mathbf{x}^{\min}$ ), and record of
           various performance metrics: percent error ( $pe$ ), number of iterations ( $k$ ),
           number of function evaluations ( $m$ );

```

Initialization step:

```

2 Normalize the search domain  $D$  to be the unit hyper-rectangle  $\bar{D}$ ;
3 Evaluate the objective function at the center point ( $\mathbf{c}^1$ ) of  $\bar{D}$  and set:
4  $\mathbf{c}^1 = (\frac{1}{2}, \frac{1}{2}, \dots, \frac{1}{2})$ ;
5  $x_j^{\min} = \lfloor b_j - a_j \rfloor c_j^1 + a_j, j = 1, \dots, n$ ; // referring to  $D$ 
6  $f^1 = f(\mathbf{x}^{\min}), f^{\min} = f^1$ ;
7 Initialize performance measures:  $k = 1, m = 1, pe$ ; //  $pe$  defined in (9)
8 while  $pe > \varepsilon_{pe}$  and  $m < M_{max}$  and  $k < K_{max}$  do
9   Selection step: Identify the set  $S_k$  of POHs using Definition 1;
10  foreach  $\bar{D}_k^j \in S_k$  do
11    Sampling step: Evaluate  $f$  at the newly sampled points in  $\bar{D}_k^j$ ;
12    Subdivision step: Trisect  $\bar{D}_k^j$  as illustrated in Fig. 1;
13  end
14  Update  $f^{\min}, \mathbf{x}^{\min}$ , and performance measures:  $k, m$  and  $pe$ ;
15 end
16 Return  $f^{\min}, \mathbf{x}^{\min}$ , and performance measures:  $k, m$  and  $pe$ .

```

Definition 1 Let $\mathbf{c}^i$ denote the center sampling point and δ_k^i be a measure (equivalently, sometimes called distance or size) of the hyper-rectangle $\bar{D}_k^i$. Let $\varepsilon > 0$ be a positive constant and $f^{\min}$ be the best currently found value of the objective function. A hyper-rectangle $\bar{D}_k^h, h \in \mathbb{I}_k$ is said to be potentially optimal if there exists some rate-of-change (Lipschitz) constant $\tilde{L} > 0$ such that

$$f(\mathbf{x}^h) - \tilde{L}\delta_k^h \leq f(\mathbf{x}^i) - \tilde{L}\delta_k^i, \quad \forall i \in \mathbb{I}_k, \quad (3)$$

$$f(\mathbf{x}^h) - \tilde{L}\delta_k^h \leq f^{\min} - \varepsilon|f^{\min}|, \quad (4)$$

where

$$x_j^i = \lfloor b_j - a_j \rfloor c_j^i + a_j, j = 1, \dots, n, \quad (5)$$

and the measure of the hyper-rectangle $\bar{D}_k^i$ is

$$\delta_k^i = \frac{1}{2} \|\mathbf{b}^i - \mathbf{a}^i\|_2. \quad (6)$$

The hyper-rectangle $\bar{D}_k^j$ is potentially optimal if the lower Lipschitz bound for the objective function computed by the left-hand side of (3) is the smallest one with some positive constant $\tilde{L}$ in the current partition $\mathcal{P}_k$. In (4), the parameter ε is used to protect from an excessive

refinement of the local minima [25, 34]. In [25], the authors obtained good results for ε values ranging from 10^{-3} to 10^{-7} . A geometrical interpretation of POH selection using Definition 1 is illustrated on the left panel of Fig. 2. Here each hyper-rectangle is represented as a point. The x -axis shows the size of the measure (δ) while the y -axis – the objective function value attained at the midpoint (c) of this hyper-rectangle. The hyper-rectangles meeting conditions (3) and (4) are points on the lower-right convex hull (highlighted in blue color).

However, such a selection strategy can be especially inefficient, e.g., for symmetric problems. There may be many POHs with the same diameter δ_k^i and objective value, leading to a drastic increase of selected POHs per iteration. To overcome this, authors in [11] proposed selecting only one of these many “equivalent” candidates. In [24], the authors revealed that such modification could significantly increase the performance of the DIRECT algorithm. In this paper, we call this an *improved original selection strategy*.

2.1.2 Original partitioning scheme

In the **Sampling** and **Subdivision** steps (see Algorithm 1, Lines 11 and 12), a hyper-rectangular partition based on n -dimensional trisection is used. Using this scheme, the POHs are partitioned into smaller non-intersecting hyper-rectangles (see Fig. 1), containing the lower function values in larger new hyper-rectangles.

2.2 Other candidate selection schemes in DIRECT-type algorithms

Various improvements and new ideas for candidate selection were proposed in the literature. To prevent the DIRECT algorithm from being sensitive to the objective function’s additive scaling, authors in [9] introduced a scaling of the objective function values by subtracting the median value calculated from the previously evaluated function values. More specifically, in the selection step, a new DIRECT- m replaces (4) from (4) to:

$$f(\mathbf{x}^i) - \tilde{L}\delta^i \leq f^{\min} - \varepsilon|f^{\min} - f^{\text{median}}|. \quad (7)$$

Similarly, in [29], authors adopted the similar idea in DIRECT- a . At each iteration, instead of the median value (f^{median}), authors proposed to use the average value (f^{average}):

$$f(\mathbf{x}^i) - \tilde{L}\delta^i \leq f^{\min} - \varepsilon|f^{\min} - f^{\text{average}}|. \quad (8)$$

The authors in [8, 28, 30] showed that different schemes controlling the ε parameter in (4) could increase the efficiency of the DIRECT algorithm, especially when needed to fine-tune the solution to higher accuracy.

In order to verify this, the experimental investigation of the original DIRECT, DIRECT- m (based on Eq. (7)), and DIRECT- a (based on Eq. (8)) algorithms on an extensive set consisting of 81 test and six engineering problems (from DIRECTGOLib v1.0 [54]) were performed in [50]. Our investigation revealed that in solving engineering problems, a significant performance difference was not observed. However, on 81 test problems, the original DIRECT proved to be more efficient, and using the same stopping conditions solved 10 and 23 more test problems than DIRECT- m and DIRECT- a accordingly (for details, see Table 3 in [50]). Based on this, the original Eq. (4) was used in an improved original selection strategy in our experimental study.

Below we focus on the other two selection schemes considered in this research.

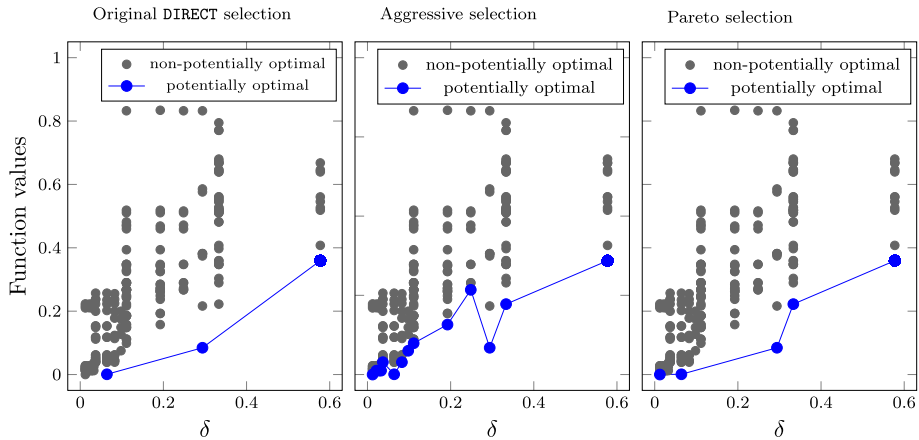


Fig. 2 Comparison of three different selection schemes (original DIRECT, aggressive, and Pareto) applied on the same set of points

2.2.1 Aggressive selection

In [1], the authors relaxed the selection criteria of POHs and proposed an aggressive version of the DIRECT algorithm (*Aggressive DIRECT*). The main idea is to select and divide at least one hyper-rectangle from each group of different diameters (δ_k^i) containing the lowest objective function value. Such aggressive selection ensures much more objective function evaluations per iteration compared to other existing POH selection schemes. From the optimization point of view, such an approach may seem less favorable since it “wastes” function evaluations by exploring unnecessary (non-potentially optimal) hyper-rectangles. However, such a strategy is much more appealing in a parallel environment, as was shown in [15, 17, 18, 58].

In [16], authors showed that limiting the refinement of the search-space when the size of hyper-rectangles δ_k^i reached some prescribed size δ^{limit} , the memory usage reduces from 10% to 70%, and the algorithm can run longer without memory allocation failure. In the experimental part (described in Sect. 4), the limit parameter (δ^{limit}) was set to the size of a hyper-rectangle that has been subdivided $50n$ times. The δ^{limit} parameter is intended for the same purpose as the equation (4) and tries to avoid wasting function estimates by “over-exploring” the local minimum area. We call this an *improved aggressive selection strategy*. A geometrical interpretation of the aggressive selection is shown in the middle panel of Fig. 2.

2.2.2 Two-step-based Pareto selection

In a more recent modification *DIRECT-GL* [52], we proposed a new two-step-based selection strategy for the identification of the extended set of POHs. In both steps, *DIRECT-GL* selects only Pareto optimal hyper-rectangles: in the first step, non-dominated on size (the higher, the better) and center point function value (the lower, the better), while in the second, non-dominated on size and distance from the current minimum point (the closer, the better) and takes the unique union of identified candidates in both steps. We note this scheme does not have any protection against over-exploration in sub-optimal local minima regions.

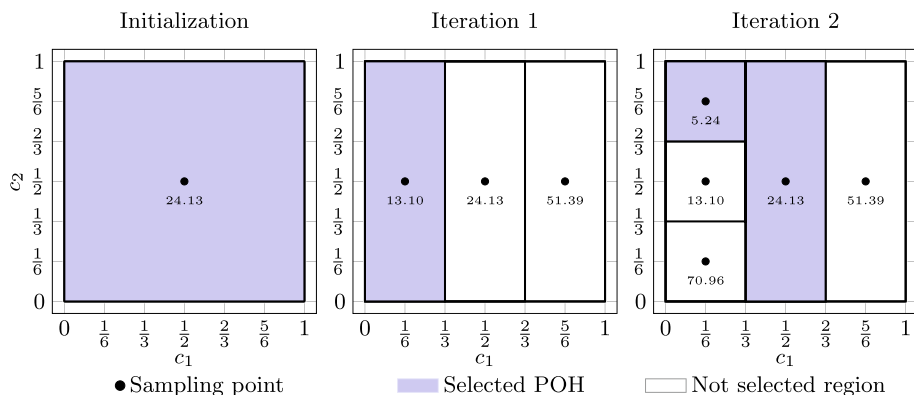


Fig. 3 Two-dimensional illustration of the partitioning technique used in the revised version of the DIRECT algorithm [23] on a two-dimensional *Bratin* test function

A geometrical interpretation of the selection procedure is shown in the right panel of Fig. 2. In the first step, DIRECT-GL selects Pareto hyper-rectangles concerning the size and function value. Therefore, unlike the Aggressive DIRECT strategy, hyper-rectangles from the groups where the minimum objective function value is higher than the minimum value from the larger groups are not selected in DIRECT-GL. Compared to the original selection (Definition 1), in DIRECT-GL, the set of POHs is enlarged by adding more medium-sized hyper-rectangles. In this sense, Pareto selection may be more global than the original DIRECT selection. Additionally, in the second step, DIRECT-GL selects the hyper-rectangles that are non-dominated concerning the size and distance from the current minimum point. This way, the set of POHs is enlarged with various size hyper-rectangles nearest the current minimum point, assuring a broader examination around it.

2.3 Other partitioning schemes in DIRECT-type algorithms

2.3.1 Trisection strategy, along single the longest side

In [23], the author proposed a revised version of the original DIRECT algorithm. One of the main modifications is to trisect selected POHs only along the single longest side (coordinate), see Fig. 3. If there are several equal longest sides, the coordinate that has been split the least times during the entire search process so far is selected. If there is a tie on the latter criterion, the lowest indexed dimension is selected. In [24], authors showed that dividing a selected rectangle on only one the longest side instead of all can significantly increase the convergence speed.

2.3.2 Diagonal trisection strategy

Adaptive diagonal curves (ADC) based algorithm was introduced in [45]. Independently of the problem dimension, the ADC algorithm evaluates the objective function $f(\mathbf{x})$ at two vertices of the main diagonals of each hyper-rectangle $\bar{D}_k^i$, as shown in Fig. 4. Same as in the revised version of DIRECT [23], each selected POH is trisected along just one of the longest sides. Such a diagonal scheme potentially obtains more comprehensive information about the

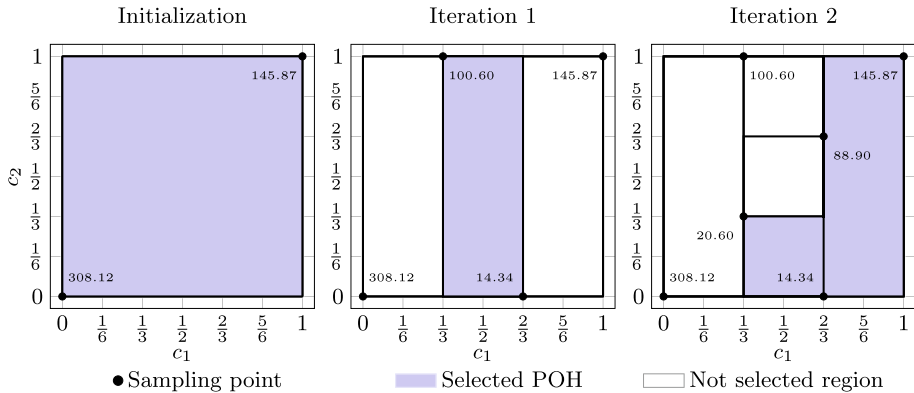


Fig. 4 Two-dimensional illustration of the diagonal trisection strategy introduced in the ADC [45] algorithm on a two-dimensional *Branin* test function

objective function than center sampling. The center sampling strategies may sometimes take many iterations to find the solution when a hyper-rectangle containing the optimum has a midpoint with a very bad function value, which makes it undesirable for further selection. The ADC algorithm intuitively reduces this chance for both sampling points in the hyper-rectangle containing the optimum solution by sampling two points per hyper-rectangle. Therefore, better performance could be expected, especially solving more complex problems.

The main advantage of such a strategy is that it addresses one of the well-known algorithmic weaknesses of the original DIRECT. The feasible region boundary points can only be approached arbitrarily closely but never sampled using the center sampling technique. Authors in [21, 28] have shown that the latter fact can cause very slow convergence to an optimum if it lies on the feasible region's boundary.

2.3.3 Diagonal bisection strategy

BIRECT (BIsecting RECTangles) [33] is motivated by the diagonal partitioning approach [45, 46, 48]. The bisection is used instead of a trisection typical for diagonal-based and most DIRECT-type algorithms. However, neither sampling at the center nor the diagonal's endpoints are appropriate for bisection. Therefore, in BIRECT, the objective function is evaluated at two points lying on the diagonal equidistant between themselves and a diagonal's vertices (see Fig. 5). Such a sampling strategy enables the reuse of the sampling points in descendant hyper-rectangles. Like the ADC algorithm (see Sect. 2.3.2), BIRECT samples two points per hyper-rectangle. Therefore, more comprehensive information about the objective function is considered compared to the central sampling strategy used in most DIRECT-type algorithms.

3 Summary of new DIRECT-type algorithmic variations

In this section, we define new variations of the DIRECT-type algorithms. In total, twelve variants of DIRECT-type algorithms are constructed (see Table 1) by combining three different selection and four partitioning techniques reviewed in the previous section.

The selection strategies used to create these new DIRECT-type algorithmic variations are:

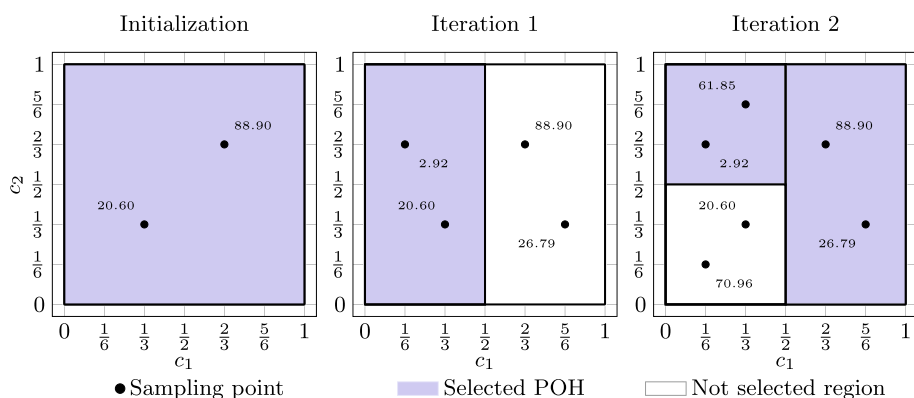


Fig. 5 Two-dimensional illustration of the diagonal bisection strategy used in the BIRECT algorithm [33] on a two-dimensional *Branin* test function

Table 1 Abbreviations of new twelve DIRECT-type algorithmic variations based on three different selection and four partitioning strategies

	Partitioning strategy			
	N-DTC	1-DTC	1-DTDV	1-DBDP
Selection scheme				
IO	N-DTC-IO	1-DTC-IO	1-DTDV-IO	1-DBDP-IO
IA	N-DTC-IA	1-DTC-IA	1-DTDV-IA	1-DBDP-IA
GL	N-DTC-GL	1-DTC-GL	1-DTDV-GL	1-DBDP-GL

1. **Improved Original selection (IO)** as described in Sect. 2.1.1.
2. **Improved Aggressive selection (IA)** as described in Sect. 2.2.1.
3. **Two-step-based (Global-Local) Pareto selection (GL)** as described in Sect. 2.2.2.

The partitioning strategies used in these combinations are the following:

1. **Hyper-rectangular partitioning based on *N*-Dimensional Trisection and objective function evaluations at Center points (N-DTC)** as described in Sect. 2.1.2.
2. **Hyper-rectangular partitioning based on 1-Dimensional Trisection and objective function evaluations at Center points (1-DTC)** as described in Sect. 2.3.1.
3. **Hyper-rectangular partitioning based on 1-Dimensional Trisection and objective function evaluations at two Diagonal Vertices (1-DTDV)** as described in Sect. 2.3.2.
4. **Hyper-rectangular partitioning based on 1-Dimensional Bisection and objective function evaluations at two Diagonal Points (1-DBDP)** as described in Sect. 2.3.3.

Let us note that some constructed combinations are already used in existing DIRECT-type algorithms. For example, the N-DTC-GL algorithm is identical to the recently proposed DIRECT-GL [52]. Furthermore, the N-DTC-IO and 1-DBDP-IO algorithms are highly related to DIRECT [25] and BIRECT [33] algorithms. The only difference is that DIRECT and BIRECT algorithms select all “equivalent” candidates (with the same diameter and objective function value), while the IO selection rule restricts to one candidate. The Aggressive DIRECT [1] algorithm is close to the N-DTC-IA variation. The only difference is the selection step using the limit parameter (δ^{limit}). Moreover, discarding the local search subroutine

from the revised hybrid version of the DIRECT algorithm [23] would lead to 1-DTC-IO variation. Finally, the 1-DTDV-IO combination is highly related to the ADC algorithm [45], but the latter approach has distinct “local” and “global” phases in the selection procedure.

4 Experimental investigation using test problems from DIRECTGOLib v1.1

Test problems from the DIRECTGOLib v1.1 library [55] (listed in “Appendix” A Table 7) are used to evaluate the developed algorithms. In total, we examined new algorithms on 96 box-constrained global optimization test instances. Note that different subsets (e.g., low dimensional problems ($n \leq 4$), non-convex problems, etc.) of the entire set were used to deepen the investigation. All problems and algorithms are implemented in the Matlab R2022a environment and are included in the most recent version of DIRECT-type Matlab toolbox DIRECTGO v1.1.0 [51]. All computations were performed on 8th Generation Intel R Core™ i7-8750H @ 2.20GHz Processor. All 12 algorithms were tested using a limit of $M_{\max} = 10^6$ function evaluations in each run. For the 96 analytical test cases with a priori known global optima f^* , the used stopping criterion is based on the percent error:

$$pe = 100\% \times \begin{cases} \frac{f(\mathbf{x}) - f^*}{|f^*|}, & f^* \neq 0, \\ f(\mathbf{x}), & f^* = 0, \end{cases} \quad (9)$$

where f^* is the known global optimum. In all experimental studies presented in this section, the algorithms were stopped when the percent error became smaller than the prescribed value $\varepsilon_{pe} = 10^{-2}$ or when the number of function evaluations exceeded the prescribed limit of 10^6 . In other words, we stop the search when the algorithm has attained an objective function value very close to the known optimum value.

Experimental results presented in this paper are also available in digital form in the Results/JOGO directory of the Github repository [51]. The Scripts/JOGO directory of the same Github repository [51] provides the MATLAB script for cycling through all different classes of DIRECTGOLib v1.1 test problems used in this paper. The constructed script can be handy for reproducing the results presented here and comparing and evaluating newly developed algorithms.

4.1 Investigation of different partitioning strategies

First, we compare the performance of new DIRECT-type algorithms by stressing the used partitioning strategy. Table 2 summarizes comparative results using all twelve DIRECT-type variations on the whole set of 96 DIRECTGOLib v1.1 test problems. In Table 2, each column corresponds to a different partitioning method. Since each partitioning method was run on the 96 problems using 3 different selection methods (rows of Table 2), it follows that each partitioning method was involved in solving $3 \times 96 = 288$ problems. The best results are marked in bold. In total, all three 1-DBDP partitioning-strategy-based algorithms failed to solve (28/288) test cases. In contrast, the second and third best partitioning technique (N-DTC, 1-DTC) based DIRECT-type approaches did not solve (29/288) and (36/288) cases accordingly. Not surprisingly, a higher number of solved test problems leads to a better overall average performance of 1-DBDP partitioning strategy-based DIRECT-type approaches. In total, the 1-DBDP partitioning technique-based DIRECT-type methods required approx-

imately 7% and 18% fewer function evaluations than the second and third best partitioning scheme (N-DTC and 1-DTC) based DIRECT-type algorithms accordingly.

However, the situation is different when comparing algorithms based on the median number of function evaluations. The diagonal trisection strategy (1-DTDV) based DIRECT-type algorithms are more efficient than competitors. The median value for all 288 test problems solved with the 1-DTDV partitioning scheme is approximately 23% and 47% better than the second and third best 1-DTC and 1-DBDP approaches accordingly. Therefore, 1-DTDV partitioning strategy-based DIRECT-type algorithms can solve at least half of these test problems with the best performance. Furthermore, most of the time, the 1-DTDV partitioning strategy-based algorithms delivered the best average results in solving low-dimensional test problems ($n \leq 4$). In total, on 153 low-dimensional ($n \leq 4$) test cases, the 1-DTDV partitioning strategy-based algorithms required approximately 22% fewer function evaluations than the second-best partition strategy (1-DTC) based variants. To sum up, the 1-DTDV partitioning strategy performs the best combined with a two-step-based selection scheme (GL). The 1-DTDV-GL algorithm in general significantly outperformed the other two variations based on IA and IO selection schemes.

The 1-DBDP partitioning strategy-based variation combined with two of the selection schemes (GL and IA) delivered the best average results on higher dimension ($n > 4$) test problems. In total, the 1-DBDP partitioning strategy-based algorithms required approximately 16% fewer function evaluations than the second-best partition strategy (N-DTC) based variants.

4.1.1 Investigating the impact of the solution on the boundary

In Sect. 2.3.2, we stressed that the diagonal trisection strategy (1-DTDV) is especially appealing on problems where the solution lies on the boundary of the feasible region. We have carried out an additional experimental study presented here to investigate this. Note that the selection strategy was fixed (IO), and only the influence on the performance of four partitioning strategies was investigated.

Out of the 46 box-constrained unique (excluding dimensionality variations) test problems from DIRECTGOLib v1.1, only the *Deb02* problem has a solution on the boundary. More precisely, the solution lies on the feasible region's vertex, making this situation particularly favorable to the 1-DTDV strategy. Experimental results using four DIRECT-type variations on the *Deb02* test problem (with varying dimensionality n) are given in the upper part of Table 3. Since the solution is at the vertex, independently of the dimension, the 1-DTDV-IO algorithm found the solution in the initialization step and took only two objective function evaluations. The other three DIRECT-type algorithms required significantly more function evaluations until the stopping condition was satisfied.

In order to carry out a more detailed investigation, test problems with solution coordinates lying on the boundary were artificially constructed. For this purpose, ten variations of 10-dimensional *Levy* and five variations of 5-dimensional *D. Price* test problems with perturbed feasible regions were created. Let us first consider the case of the *Levy* function. The original feasible region is $D = [-5, 5]^{10}$ and the solution point $\mathbf{x}^* = (1, 1, \dots, 1)$. Thus none of the solution coordinates are on the boundary of the permissible area. On the original *Levy* problem, the 1-DTDV-IO algorithm performed significantly worse than any of the other three tested DIRECT-type counterparts (see the first row in the middle part of Table 3).

However, the situation changes completely when *Levy* variations with the perturbed feasible region are considered. We artificially reconstructed the original domain so that an

Table 2 The number of function evaluations of twelve DIRECT-type variants on DIRECTGOLib v1.1 test problems

Criteria/algorithms	# of cases	N-DTC-IA	1-DTC-IA	1-DBDP-IA	1-DTDV-IA
# of failed problems	96	13	13	11	18
Average results	96	172, 805	160, 691	146, 887	202, 694
Average ($n \leq 4$)	51	25, 968	23, 638	45, 643	9, 785
Average ($n > 4$)	45	339, 791	316, 541	262, 640	421, 539
Average (convex)	30	149, 711	126, 030	109, 374	153, 594
Average (non-convex)	66	183, 302	176, 446	163, 939	225, 012
Average (uni-modal)	15	108, 068	78, 226	73, 957	111, 805
Average (multi-modal)	81	187, 744	179, 722	163, 717	223, 668
Median results	96	7, 608	1, 287	2, 108	1, 586
Criteria/algorithms	# of cases	N-DTC-IO	1-DTC-IO	1-DBDP-IO	1-DTDV-IO
# of failed problems	96	12	18	12	21
Average results	96	142, 277	211, 463	146, 133	227, 455
Average ($n \leq 4$)	51	43, 832	42, 633	41, 602	41, 990
Average ($n > 4$)	45	254, 819	403, 749	265, 522	438, 574
Average (convex)	30	111, 817	170, 675	80, 490	171, 868
Average (non-convex)	66	156, 122	230, 004	175, 971	252, 722
Average (uni-modal)	15	60, 100	57, 360	62, 016	111, 547
Average (multi-modal)	81	161, 240	247, 026	165, 545	254, 203
Median results	96	771	1, 198	953	847
Criteria/Algorithms	# of cases	N-DTC-GL	1-DTC-GL	1-DBDP-GL	1-DTDV-GL
# of failed problems	96	4	5	5	5
Average results	96	71, 488	62, 475	65, 442	71, 319
Average ($n \leq 4$)	51	9, 675	7, 073	41, 300	5, 772
Average ($n > 4$)	45	141, 753	125, 417	93, 714	145, 733
Average (convex)	30	55, 320	45, 520	42, 326	8, 950
Average (non-convex)	66	78, 837	70, 182	75, 949	99, 669
Average (uni-modal)	15	28, 478	12, 624	23, 300	25, 796
Average (multi-modal)	81	81, 183	73, 979	75, 398	81, 825
Median results	96	1, 848	960	2, 042	775

increasing number of solution coordinates are located on the boundary—the column “Feasible region” in Table 3 specifies the modified feasible region. For example, for the first *Levy* variation (*Levy1*), the modified feasible region coincides with the original one (i.e., $D_1^m = D$) apart from the coordinate x_1 , whose new domain is $x_1 \in [-5, 1]$ (original was $x_1 \in [-5, 5]$). Therefore, for the *Levy1* problem, one of its solution coordinates (x_1) is on the boundary of the modified feasible region. The other nine *Levy* function variations are again obtained by substituting only one but the following coordinate compared to the previous *Levy* variation, as shown in Table 3. The ν value (the third column in Table 3) indicates the number of solution coordinates projected onto the boundary. From the obtained results presented in Table 3, observe that the more coordinates of the solution are located on the boundary, the

fewer objective function evaluations the 1-DTDV-IO algorithm needs to find the solution. When $\nu \geq 8$ (i.e., at least eight out of 10 coordinates lie on the boundary), the 1-DTDV-IO algorithm outperformed other approaches. However, the situation is the opposite of the other three partitioning schemes based on DIRECT-type algorithms. For almost all cases, they required more function evaluations when ν increases.

In the last investigation, five variations of *D. Price* test function (original $D = [-10, 10]^5$ and the solution point lying close to the center-point of the domain) were considered. For *D. Price*, we perturbed the feasible region by the same strategy as for the *Levy* test problem. Same as before, when most of the solution coordinates are located on the boundaries of the domain ($\nu \geq 4$), the 1-DTDV-IO algorithm is the most efficient. However, unlike for the *Levy* function, the presence of the solution coordinates on the boundary did not worsen the other DIRECT-type algorithms but rather improved. These results show that the performance of center-based partitioning techniques will not necessarily worsen when at least part of the solution coordinates are on the boundary.

4.1.2 Investigating the impact of the domain perturbation

By investigating the impact of different partitioning strategies, we observed several situations where one method (or partitioning scheme) dominates the others significantly. However, sometimes one method may be lucky because the partitioning approach in the initial steps naturally samples near the solution. In such situations, the location of the solution may favor one partitioning scheme over another. In this section, we explore whether such dominance is robust to slight perturbations of the domain.

Initially, we identified test problems for which a particular partitioning scheme (regardless of the selection strategy) had a clear dominance, possibly due to the conveniently defined variable bounds. Out of the 46 DIRECTGOLib v1.1 unique box-constrained test problems, the dominance of a particular partitioning scheme was identified for eight of them. We made domain perturbations for all eight problems keeping the same optimal solution. First, if any partitioning scheme-based DIRECT-type algorithm can find the solution in the initialization step, the original domain (D) has been shifted by 22.5% to the right side. In some cases, we have made further perturbations that none of the partitioning schemes would sample on initial iterations close to the solution (the column “Feasible region” in Table 4 specifies the original and perturbed domains).

The obtained experimental results revealing the impact of the domain perturbation of twelve DIRECT-type variants on eight selected original and perturbed test problems with varying dimensionality are given in Table 4. None of the partitioning schemes proved to be robust. Perturbation of the bounds may significantly reduce the dominance of a particular partitioning scheme. Quite often, the previously dominant approach may not solve the perturbed problem within the given budget of function evaluations at all. For example, the 1-DTC partitioning scheme-based DIRECT-type algorithms undoubtedly dominate original *Rastrigin* and *Griewank* test problems. However, for perturbed variations of *Rastrigin* and *Griewank*, the same 1-DTC partitioning scheme-based DIRECT-type algorithms could not be able to find a solution for most of these cases.

Another obvious example is the *Rosenbrock* test problem case. The 1-DTDV partitioning scheme-based DIRECT-type algorithms are most efficient when the problem is considered in the original domain D . However, after the D 's perturbation, the 1-DTDV partitioning scheme proved to be very inefficient. Also, in some cases, the bounds' perturbation helped other algorithms perform significantly better than on the original domain. Examples of such problems are *Styblinski-Tang*, *Easom*, and *Power Sum*. Furthermore, only for the *Schweffel* test

Table 3 The number of function evaluations required for four different DIRECT-type algorithms to find optimal solution lying on the boundary

Label	n	ν	Feasible region	N-DTC-IO	1-DTC-IO	1-DBDP-IO	1-DTDV-IO
<i>Deb02</i>	2	2	$D = [0, 1]^n$	77	75	100	2
<i>Deb02</i>	4	4	$D = [0, 1]^n$	199	145	220	2
<i>Deb02</i>	8	8	$D = [0, 1]^n$	653	329	494	2
<i>Deb02</i>	16	16	$D = [0, 1]^n$	3,827	1,279	2,368	2
<i>Levy</i>	10	0	$D = [-5, 5]^n$	2,589	919	1,496	141,999
<i>Levy1</i>	10	1	$D_1^m = D$ and $x_1 \in [-5, 1]$	2,847	973	1,326	24,439
<i>Levy2</i>	10	2	$D_2^m = D_1^m$ and $x_2 \in [1, 5]$	3,221	1,033	1,386	16,291
<i>Levy3</i>	10	3	$D_3^m = D_2^m$ and $x_3 \in [-10, 1]$	3,447	1,079	2,002	13,625
<i>Levy4</i>	10	4	$D_4^m = D_3^m$ and $x_4 \in [1, 10]$	3,919	1,119	2,048	14,166
<i>Levy5</i>	10	5	$D_5^m = D_4^m$ and $x_5 \in [-2, 1]$	4,091	1,195	2,116	10,927
<i>Levy6</i>	10	6	$D_6^m = D_5^m$ and $x_6 \in [1, 4]$	4,483	1,287	2,316	5,416
<i>Levy7</i>	10	7	$D_7^m = D_6^m$ and $x_7 \in [-7, 1]$	5,215	2,193	2,484	3,069
<i>Levy8</i>	10	8	$D_8^m = D_7^m$ and $x_8 \in [1, 15]$	5,487	2,579	3,174	2,494
<i>Levy9</i>	10	9	$D_9^m = D_8^m$ and $x_9 \in [-13, 1]$	6,299	6,581	3,518	547
<i>Levy10</i>	10	10	$D_{10}^m = D_9^m$ and $x_{10} \in [1, 10]$	6,487	2,487	3,572	551
<i>D. Price</i>	5	0	$D = [-10, 10]^n$	22,465	20,791	4,060	134,011
<i>D. Price1</i>	5	1	$D_1^m = D$ and $x_1 \in [-19, 1]$	18,245	18,707	2,930	16,089
<i>D. Price2</i>	5	2	$D_2^m = D_1^m$ and $x_2 \in [0.7071, 21]$	4,975	1,455	1,322	3,434
<i>D. Price3</i>	5	3	$D_3^m = D_2^m$ and $x_3 \in [-19, 0.5946]$	7,709	3,845	1,610	2,759
<i>D. Price4</i>	5	4	$D_4^m = D_3^m$ and $x_4 \in [0.5452, 21]$	3,989	1,315	2,280	728
<i>D. Price5</i>	5	5	$D_5^m = D_4^m$ and $x_5 \in [-19, 0.5221]$	3,247	1,443	2,396	565

 ν —the number of solution coordinates lying on the boundary

problem, the same two partitioning schemes (1-DBDP and 1-DTDV) based DIRECT-type algorithms remained the most efficient in the original and perturbed domains.

4.2 Investigation of different selection schemes

Here, the efficiency of new DIRECT-type algorithms is investigated based on the selection scheme. In Table 2, three row parts corresponds to a different selection approach. Since each selection strategy was run on the 96 problems using 4 different partitioning methods (columns of Table 2), it follows that each selection approach was involved in solving $4 \times 96 = 384$ test problems. We note that algorithms incorporating a two-step-based Pareto selection scheme (GL) combined with any partitioning strategy, on average, deliver the best results. All algorithmic variants based on the GL selection scheme did not solve (19/384). In contrast, the IO and IA selection schemes based algorithms failed to solve (63/384) and (55/384) cases accordingly. This leads to a much better average performance of DIRECT-type algorithms based on the GL selection scheme. In total, GL selection scheme-based algorithms required approximately 60% and 62% fewer function evaluations compared with the IO and IA counterparts. The GL selection scheme's most significant advantage can be seen in solving higher-dimensional ($n > 4$) test problems. In total, GL selection scheme-based algorithms solving ($n > 4$) test instances required approximately 72% fewer function evaluations compared with the IO or IA selection scheme-based counterparts.

The IO selection scheme seems the most suitable for simpler optimization problems (low dimensional, uni-modal, and problems with a few minima). Of 63 failed problems using the IO selection scheme, 43 were extremely hard, i.e., multi-modal, sharply peaked, and multi-variable (e.g., $n \geq 10$). The GL selection strategy-based variants usually select more regions to subdivide. Therefore, they suffer for these more straightforward optimization problems. However, the GL scheme ensured, that on average, all DIRECT-type variants converged in significantly fewer function evaluations on complex multi-modal test problems.

Additionally, the operational characteristics [14, 56] reported in Figs. 6, 7, 8 show the behavior of all twelve algorithms on different subsets of DIRECTGOLib v1.1 box-constrained test problems. Operational characteristics provide the proportion of problems that can be solved within a given budget of function evaluations. Figure 6, drawn using all 96 box-constrained problems from DIRECTGOLib v1.1, clearly shows that IO selection scheme-based DIRECT-type algorithms (1-DTDV-IO and N-DTC-IO) dominate for simpler problems. They solved almost half of the 96 test problems within a small budget of objective function evaluations. However, as the number of function evaluations increases (as more complex problems are considered), the GL scheme-based algorithms are most efficient.

Operational characteristics in Fig. 7 show the behavior of all twelve algorithms on 35 higher-dimensionality ($n > 4$) multi-modal test problems. Once again, when a given budget of function evaluations is low ($M_{\max} \leq 500$), all IO selection scheme-based variations perform better. Unfortunately, with such a small function evaluation budget, the algorithms will only solve approximately 15% of all the test problems. When the maximal budget of function evaluations increased ($M_{\max} \leq 4,000$), only one of the IO selection scheme combinations (N-DTC-IO) maintained the highest efficiency and solved approximately 50% of all the test cases. Finally, when the function evaluation budget is higher ($M_{\max} \geq 4,000$), GL selection scheme-based variations (1-DTC-GL and 1-DBDP-GL) have the highest efficiency.

Similar tendencies regarding the best-performing selection strategies can be seen in Fig. 8. Here, the operational characteristics illustrate the behavior of algorithms solving simplest

Table 4 Experimental results of twelve DIRECT-type variants on 8 selected original and perturbed test problems with varying dimensionality

Label	n	Feasible region	N-DTC-1A	1-DTC-1A	1-DBDP-1A	1-DTDV-1A	N-DTC-1A	1-DTC-1A	1-DBDP-1A	1-DTDV-1A	N-DTC-GL	1-DTC-GL	1-DBDP-GL	1-DTDV-GL
<i>Alpine</i>	5	$[0, 10]^n$	61, 485	10, 343	714	$> 10^6$	2, 033	1, 231	168	460, 445	1, 287	685	320	27, 074
	10	$[0, 10]^n$	$> 10^6$	$> 10^6$	173, 912	$> 10^6$	$> 10^6$	$> 10^6$	$> 10^6$	$> 10^6$	61, 209	4, 063	7, 646	173, 948
<i>Perturbed Alpine</i>	5	$[\frac{1}{\sqrt{2}}, 8 + \frac{1}{\sqrt{2}}]^n$	61, 485	10, 343	2, 678	10, 941	2, 209	1, 403	5, 790	3, 920	1, 479	853	3, 462	19, 417
	10	$[\frac{1}{\sqrt{2}}, 8 + \frac{1}{\sqrt{2}}]^n$	$> 10^6$	$> 10^6$	218, 170	$> 10^6$	$> 10^6$	$> 10^6$	$> 10^6$	$> 10^6$	31, 967	7, 245	15, 500	$> 10^6$
<i>Griewank</i>	5	$[-330, 870]^n$	719, 985	69, 979	$> 10^6$	47, 581	$> 10^6$	$> 10^6$	$> 10^6$	$> 10^6$	524, 765	457, 207	14, 706	$> 10^6$
	10	$[-330, 870]^n$	$> 10^6$	8, 799	20, 534	$> 10^6$	$> 10^6$	$> 10^6$	$> 10^6$	$> 10^6$	$> 10^6$	14, 593	204, 940	$> 10^6$
<i>Perturbed Griewank</i>	5	$\left[-\sqrt{600i}, \frac{600}{\sqrt{i}}\right]^n$	$> 10^6$	$> 10^6$	$> 10^6$	$> 10^6$	$> 10^6$	$> 10^6$	$> 10^6$	$> 10^6$	187, 811	102, 961	175, 806	353, 028
	10	$\left[-\sqrt{600i}, \frac{600}{\sqrt{i}}\right]^n$	$> 10^6$	$> 10^6$	$> 10^6$	$> 10^6$	$> 10^6$	$> 10^6$	$> 10^6$	$> 10^6$	$> 10^6$	$> 10^6$	$> 10^6$	$> 10^6$
<i>Syblinski-Tang</i>	5	$[-5, 5]^n$	4, 941	627	164	243, 882	539	$> 10^6$	78	7, 077	1, 779	865	192	682
	10	$[-5, 5]^n$	68, 025	2, 631	714	$> 10^6$	9, 785	$> 10^6$	180	$> 10^6$	11, 347	3, 237	784	5, 248
<i>Perturbed Syblinski-Tang</i>	5	$[-5, 5 + \frac{1}{\sqrt{3}}]^n$	3, 919	533	1, 056	66, 810	395	273	278	4, 098	1, 659	841	1, 654	36, 655
	10	$[-5, 5 + \frac{1}{\sqrt{3}}]^n$	68, 025	2, 151	6, 736	$> 10^6$	2, 917	829	1, 368	$> 10^6$	13, 431	3, 447	11, 386	$> 10^6$
<i>Easom</i>	2	$[-100, 100]^n$	433, 031	429, 743	444	20, 316	7, 581	6, 619	322	6, 651	451	321	544	348
<i>Perturbed Easom</i>	2	$\left[\frac{-100}{i+1}, 100\right]^n$	214, 331	429, 743	71, 392	177, 258	3, 689	6, 659	18, 462	10, 078	475	393	924	376

Table 4 continued

Label	n	Feasible region	N-DTC-1A	1-DTC-1A	1-DBDP-1A	1-DTDV-1A	N-DTC-1O	1-DTC-1O	1-DBDP-1O	1-DTDV-1O	N-DTC-GL	1-DTC-GL	1-DBDP-GL	1-DTDV-GL
<i>Power Sum</i>	4	$[0.9, 4.9]^n$	$> 10^6$	$> 10^6$	13, 828	$> 10^6$	321, 595	144, 385	4, 790	$> 10^6$	69, 327	77, 353	14, 214	37, 012
<i>Perturbed Power Sum</i>	4	$[1, 5 + \sqrt[4]{2}]^n$	502, 981	176, 843	78, 746	59, 390	67, 959	25, 453	17, 930	12, 219	152, 083	70, 745	12, 494	40, 753
<i>Rastrigin</i>	5	$[-2.75, 7.25]^n$	8, 703	1, 487	314, 712	$> 10^6$	597	453	38, 714	112, 597	2, 721	1, 895	19, 642	7, 345
	10	$[-2.75, 7.25]^n$	143, 755	7, 215	$> 10^6$	$> 10^6$	4, 299	1, 551	$> 10^6$	$> 10^6$	22, 971	8, 105	$> 10^6$	140, 756
<i>Perturbed Rastrigin</i>	5	$[-5\sqrt[4]{2}, 7 + \sqrt[4]{2}]^n$	$> 10^6$	$> 10^6$	$> 10^6$	$> 10^6$	$> 10^6$	567, 269	694, 812	$> 10^6$	73, 727	24, 119	16, 440	90, 134
	10	$[-5\sqrt[4]{2}, 7 + \sqrt[4]{2}]^n$	$> 10^6$	$> 10^6$	$> 10^6$	$> 10^6$	$> 10^6$	$> 10^6$	$> 10^6$	$> 10^6$	661, 971	$> 10^6$	$> 10^6$	$> 10^6$
<i>Rosenbrock</i>	5	$[-5, 10]^n$	73, 485	26, 325	3, 208	1, 471	15, 577	1, 889	1, 494	916	26, 891	15, 695	5, 110	1, 568
	10	$[-5, 10]^n$	297, 755	$> 10^6$	13, 366	4, 541	71, 021	$> 10^6$	4, 590	2, 091	104, 643	171, 019	22, 194	5, 759
<i>Perturbed Rosenbrock</i>	5	$\left[-\frac{5}{\sqrt{i}}, 10\sqrt{i}\right]^n$	434, 985	385, 979	291, 612	$> 10^6$	55, 693	21, 363	101, 508	$> 10^6$	27, 763	7, 795	33, 056	16, 971
	10	$\left[-\frac{5}{\sqrt{i}}, 10\sqrt{i}\right]^n$	$> 10^6$	$> 10^6$	$> 10^6$	$> 10^6$	$> 10^6$	$> 10^6$	$> 10^6$	$> 10^6$	383, 081	185, 325	316, 392	432, 903
<i>Schweffel</i>	5	$[-500, 500]^n$	$> 10^6$	368, 479	7, 566	$> 10^6$	74, 989	16, 767	1, 070	9, 561	768, 549	49, 247	4, 842	109, 746
	10	$[-500, 500]^n$	$> 10^6$	$> 10^6$	817, 512	$> 10^6$	$> 10^6$	$> 10^6$	$> 10^6$	$> 10^6$	$> 10^6$	$> 10^6$	33, 522	$> 10^6$
<i>Perturbed Schweffel</i>	5	$\left[-500 + \frac{100}{\sqrt{i}}, 500 - \frac{40}{\sqrt{i}}\right]^n$	$> 10^6$	458, 979	19, 972	2, 135	80, 295	35, 091	84, 096	33, 622	336, 581	65, 329	9, 548	1, 580
	10	$\left[-500 + \frac{100}{\sqrt{i}}, 500 - \frac{40}{\sqrt{i}}\right]^n$	$> 10^6$	$> 10^6$	$> 10^6$	$> 10^6$	$> 10^6$	$> 10^6$	$> 10^6$	$> 10^6$	$> 10^6$	$> 10^6$	99, 824	336, 983

$i = 1, \dots, n$ —indexes used for variable bounds

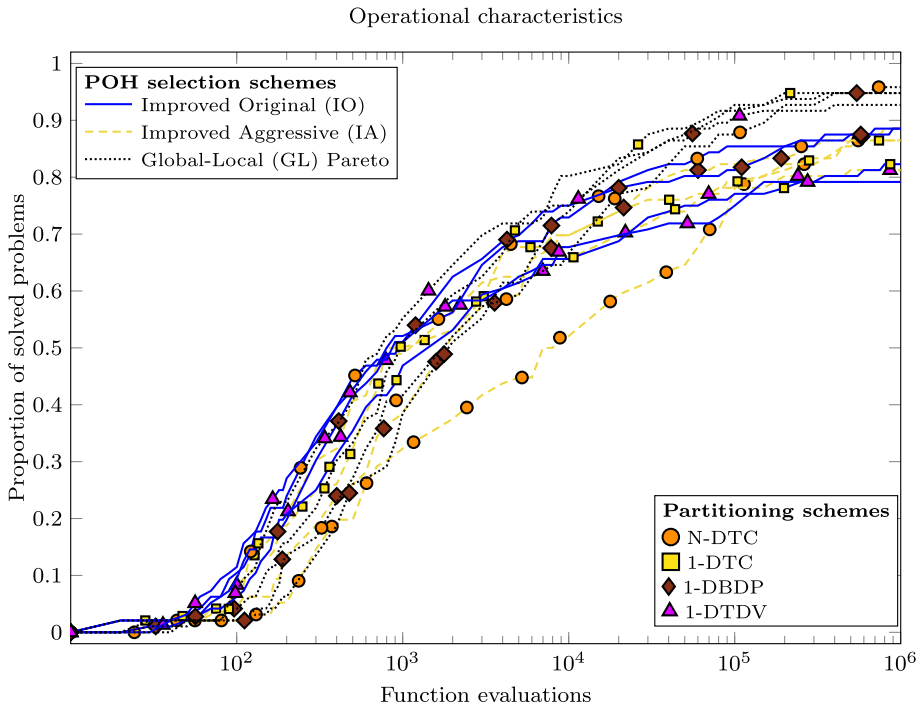


Fig. 6 Operational characteristics for all twelve DIRECT-type algorithmic variations on DIRECTGOLib v1.1 test problems

uni-modal and convex optimization test problems. Among the partitioning strategies, the 1-DTDV scheme looks the most efficient here.

5 Experimental investigation using GKLS-type test problems

Additionally, we compare the performance of all twelve DIRECT-type variants on GKLS-type test problems [13]. GKLS-generator allows generating three types (non-differentiable, continuously differentiable, and twice continuously differentiable) of multi-dimensional and multi-extremal optimization test functions with a priori known local and global minima. The complexity of generated problems is established by setting different values for user-determined parameters: problem dimension n , the number of local minima m , global minimum value f^* , distance d from the global minimizer to the paraboloid vertex, and radius r of the attraction region of the global minimizer.

We use eight different complexity classes (see Table 5). The dimensionality (n) and other parameters are set as in [34, 35]. Each class consisted of 100 test instances. For each dimension n , two test classes were considered: the “simple” class and the “hard” one. For three- and four-dimensional classes the difficulty is increased by enlarging the distance d from the global minimizer ($\mathbf{x}^*$) to the paraboloid vertex. For two and five-dimensional classes this is achieved by decreasing the radius r of the attraction region of the global minimizer.

The same stopping rule is adopted in these experiments as in [34, 35]. The global minimizer $\mathbf{x}^* \in D$ is considered to be found when an algorithm generated a function evaluation point

Operational characteristics

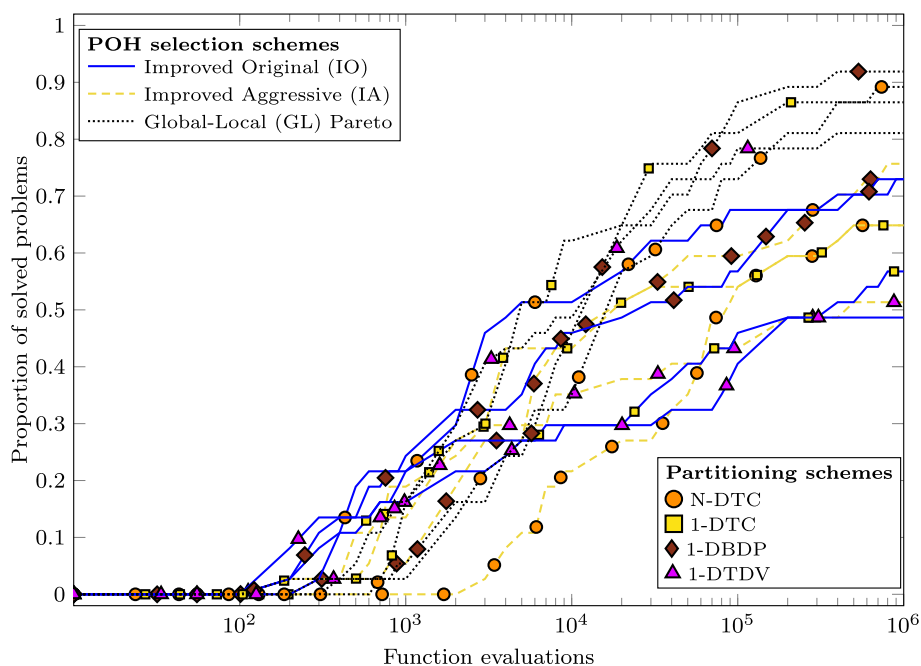


Fig. 7 Operational characteristics for all twelve DIRECT-type algorithmic variations solving higher-dimensional ($n > 4$) multi-modal DIRECTGOLib v1.1 test problems

$\mathbf{x}^i \in D_k^i$ such that:

$$|x_j^i - x_j^*| \leq \sqrt[n]{\Delta}(b_j - a_j), \quad 1 \leq j \leq n, \quad (10)$$

where $0 \leq \Delta \leq 1$ is an accuracy coefficient [45] (see Table 5 for Δ parameter values). In other words, we stop the search when the algorithm has produced a point very close to the known optimum. In each run, we used the same limit of function evaluations equal to 10^6 . Note that the stopping rule (10) does not require algorithms to find a solution with high accuracy. Therefore, the Pareto selection enhancing the local search (see Sect. 2.2.2) was disabled.

The experimental results are summarized in Table 6. The notation “ $> 10^6(j)$ ” indicates that after the maximal number of function evaluations 10^6 , the algorithm under consideration was not able to solve j problems in total. First, contrary to the previous tendencies, the best results are obtained when DIRECT-type variants include an improved original selection scheme (IO). In numerical experiments described in Sect. 4, DIRECT-type algorithms with integrated IO selection schemes had almost the worst efficiencies. However, all four DIRECT-type variants based on the IO selection scheme are promising for GKLS-type test problems. The least attractive is an improved aggressive selection scheme (IA). All failed 40 cases appeared when this selection scheme was combined with three different partitioning strategies (except 1-DBDP).

The best average results (see the upper part of Table 6) are achieved using 1-DTC-IO (for seven different classes) and 1-DBDP-GL (for one class). The average number using the 1-DTC-IO algorithm is 8,021, while the second (1-DBDP-IO) and third-best (1-DTDV-IO)

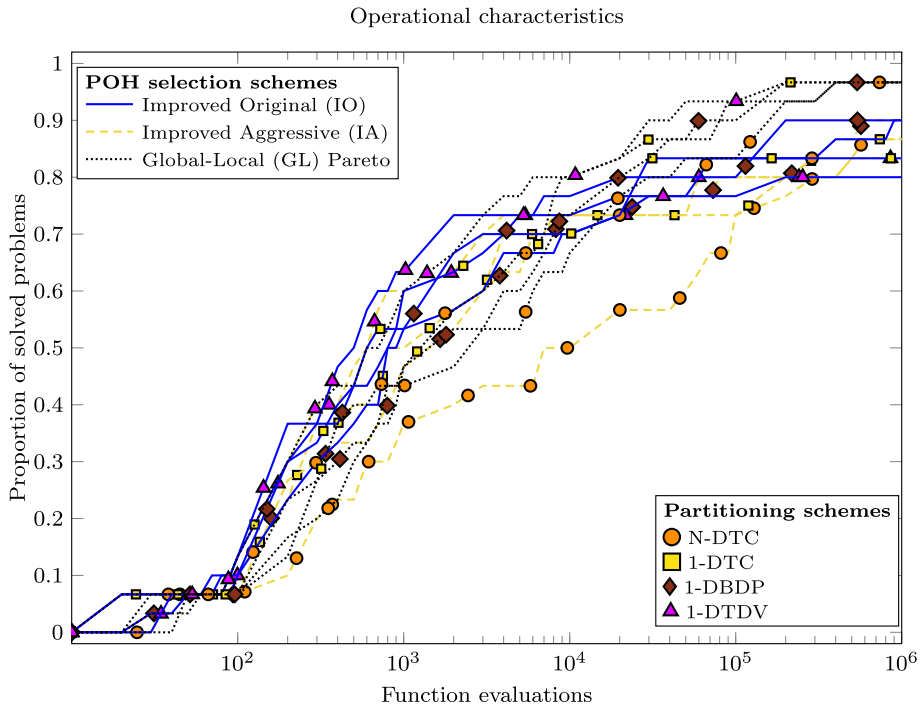


Fig. 8 Operational characteristics for all twelve DIRECT-type algorithmic variations solving uni-modal and convex DIRECTGOLib v1.1 test problems

Table 5 Description of GKLS-type test classes used in numerical experiments

Class	Difficulty	Δ	n	f^*	d	r	m
1	Simple	10^{-4}	2	-1	0.90	0.20	10
2	Hard	10^{-4}	2	-1	0.90	0.10	10
3	Simple	10^{-6}	3	-1	0.66	0.20	10
4	Hard	10^{-6}	3	-1	0.90	0.20	10
5	Simple	10^{-6}	4	-1	0.66	0.20	10
6	Hard	10^{-6}	4	-1	0.90	0.20	10
7	Simple	10^{-7}	5	-1	0.66	0.30	10
8	Hard	10^{-7}	5	-1	0.66	0.20	10

algorithms deliver approximately 19% (9, 571) and 47% (15, 231) worse overall performances. Interestingly, the 1-DBDP-GL algorithm, which produced the best overall result in Sect. 4, ranks only fifth as delivered 67% (24, 746) worse average results than 1-DTC-IO.

The lowest aggregated median number (the middle part of Table 6) for all eight classes again is obtained using the same 1-DTC-IO algorithm (1, 427). Therefore, the 1-DTC-IO algorithm can solve at least half of GKLS-type problems with the best performance. In contrast, the second (1-DBDP-GL) and third (1-DBDP-IO) best algorithms delivered approximately 3.71% (1, 482) and 3.84% (1, 484) worse overall median values.

Table 6 Comparison of twelve DIRECT-type variants on eight classes of GKLS-type problems

Class	N-DTC-IA	1-DTC-IA	1-DBDP-IA	1-DTDV-IA	N-DTC-IA	1-DTC-IO	1-DBDP-IO	1-DTDV-IO	N-DTC-GL	1-DTC-GL	1-DBDP-GL	1-DTDV-GL
Average number of function evaluations												
1	510	289	257	1148	198	149	156	360	236	217	185	665
2	5, 215	5, 491	1, 449	6, 278	1, 068	803	863	1, 458	2, 332	3, 130	1, 245	3, 454
3	4, 069	1, 747	1, 252	9, 870	1, 019	673	869	1, 776	1, 264	1, 286	825	4, 148
4	15, 331	9, 393	4, 032	35, 291	2, 477	1, 543	1, 832	4, 539	5, 125	4, 427	2, 640	14, 943
5	36, 586	20, 911	13, 179	91, 161	7, 843	4, 591	8, 207	11, 806	10, 720	10, 553	9, 199	32, 597
6	198, 129	157, 187	95, 541	350, 033	26, 970	16, 899	23, 905	34, 020	77, 715	68, 348	57, 867	117, 684
7	21, 811	17, 769	4, 956	67, 634	6, 216	4, 419	3, 974	11, 156	12, 262	10, 645	3, 472	32, 108
8	308, 948	226, 208	96, 621	477, 868	67, 636	35, 091	36, 768	55, 625	120, 118	99, 362	58, 278	183, 881
1 – 8	73, 825	54, 874	27, 160	129, 910	14, 178	8, 021	9, 571	15, 231	27, 783	24, 746	16, 713	48, 685
Median number of function evaluations												
1	308	207	204	878	119	122	111	328	128	155	130	469
2	4, 316	2, 375	1, 016	5, 793	1, 058	724	673	1, 364	1, 979	2, 097	855	3, 272
3	1, 267	949	755	7, 813	387	503	488	1, 597	445	604	461	3, 344
4	8, 970	3, 962	2, 366	32, 915	1, 782	1, 075	1, 189	4, 230	2, 463	2, 992	1, 399	13, 753
5	15, 523	9, 063	5, 851	78, 151	4, 874	2, 872	4, 443	10, 761	4, 388	7, 052	4, 202	28, 822
6	121, 285	76, 586	55, 130	334, 277	15, 517	9, 237	15, 628	32, 796	43, 458	33, 804	29, 189	116, 530
7	7, 514	4, 079	2, 102	42, 804	1, 673	2, 291	2, 278	10, 992	1, 533	3, 440	1, 427	29, 635
8	203, 711	128, 408	53, 291	429, 811	43, 400	24, 327	19, 967	47, 221	65, 892	54, 497	27, 697	162, 411
1 – 8	6, 767	4, 188	2, 098	26, 227	1, 644	1, 427	1, 487	4, 599	2, 117	3, 178	1, 482	13, 235

Table 6 continued

Class	N-DTC-IA	1-DTC-IA	1-DBDP-IA	1-DTDV-IA	N-DTC-IO	1-DTC-IO	1-DBDP-IO	1-DTDV-IO	N-DTC-GL	1-DTC-GL	1-DBDP-GL	1-DTDV-GL
Maximal number of function evaluations												
1	4, 777	1, 955	1, 178	4, 811	1, 153	655	840	961	2, 031	1, 319	1, 090	2, 502
2	21, 841	25, 021	11, 674	20, 089	3, 197	2, 201	4, 374	3, 964	8, 431	11, 041	8, 836	10, 269
3	31, 291	13, 233	8, 196	34, 607	6, 625	3, 273	5, 032	4, 864	10, 723	11, 335	5, 058	16, 789
4	132, 121	150, 809	22, 720	88, 327	15, 307	9, 763	7, 806	10, 236	48, 371	45, 721	14, 064	37, 514
5	212, 339	131, 783	116, 136	280, 015	39, 129	18, 853	62, 016	35, 898	74, 277	47, 527	70, 028	78, 365
6	$> 10^6(4)$	$> 10^6(4)$	562, 156	932, 395	260, 793	126, 061	141, 914	86, 105	907, 497	662, 983	345, 082	280, 069
7	266, 007	220, 647	77, 998	317, 351	110, 237	33, 691	27, 380	41, 751	86, 269	135, 889	54, 400	145, 022
8	$> 10^6(8)$	$> 10^6(6)$	776, 052	$> 10^6(18)$	472, 125	229, 583	313, 420	210, 483	960, 573	700, 615	436, 072	718, 803

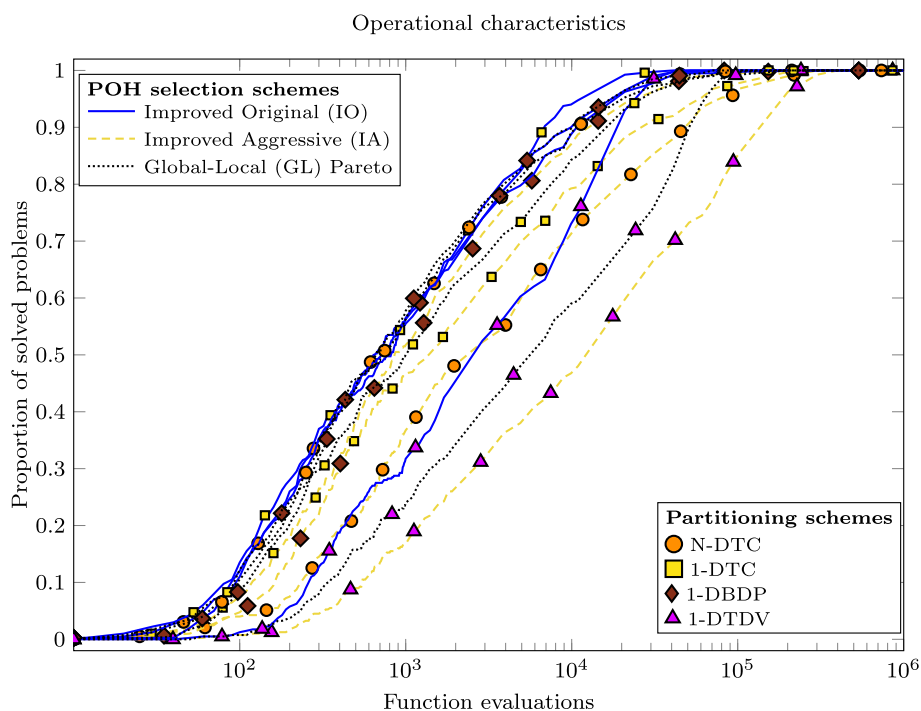


Fig. 9 Operational characteristics for all twelve DIRECT-type algorithmic variations on four “simple” GKLS-type classes

In the bottom part of Table 6, the maximal number of function evaluations required to solve test problems within a particular class is reported. The previously emphasized algorithmic variation 1-DTC-IO is the best for four out of eight classes. However, solving two of the most complex (“hard”) classes (No = 6 and 8) 1-DTDV-IO algorithm seems the most promising.

Additionally, we visualize the performance of all twelve DIRECT-type variations on GKLS-type problems using the operational characteristics. Fig. 9 shows the behavior on four “simple” GKLS classes, while Fig. 10 shows the “hard” ones. For simple classes, IO and GL selection schemes seem the most promising. Among algorithms, when a low budget of function evaluations is considered ($M_{\max} \leq 400$), the 1-DTC-IO algorithm is the most efficient. However, when $M_{\max} > 400$, the 1-DBDP-GL algorithmic combination outperforms all others. Furthermore, the best performance on these simple classes is achieved regardless of the selection scheme used with the 1-DBDP partitioning strategy.

Finally, for “hard” classes (see Fig. 10) IO selection scheme seems the most favorable (especially for simpler problems), while GL is the second-best option. However, when a higher maximal number of function evaluations is allowed, the performance of GL and IO selection scheme-based algorithms is quite similar. Among the algorithms, 1-DBDP-IO and 1-DTC-IO are the two best-performing ones.

6 Conclusions and future work

This paper presented an extensive experimental investigation of various candidate selection and partitioning techniques traditionally used in the DIRECT-type algorithms. Twelve DIRECT-type algorithmic combinations were created by considering four well-known partitioning and three selection schemes. In general, experimental results confirmed the

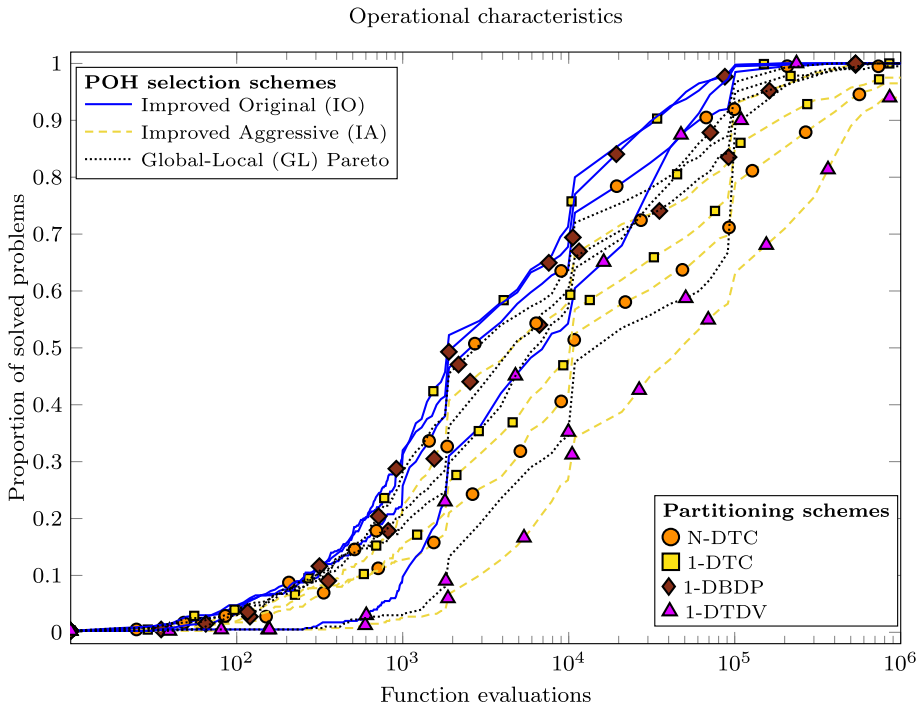


Fig. 10 Operational characteristics for all twelve DIRECT-type algorithmic variations on four “hard” GKLS-type classes

well-known fact from “No free lunch theorems for optimization” [59] that no one particular optimization algorithm works best for every problem. However, detailed experimental studies have helped identify particular DIRECT-type algorithmic variations that work well in certain situations. For example, our experimental findings in Sect. 4.1.2 revealed that what initially looks like a clear dominance case goes away with small domain perturbations. This should remind us how dangerous it can be to generalize from limited test-function results. Below, we emphasize when certain variations have performed best and make some recommendations based on that.

Investigation using DIRECTGOLib v1.1 test problems showed that independently on the partitioning strategy, a two-step-based Pareto selection scheme (GL) ensures the best performance on more challenging optimization problems (higher-dimensionality, multi-modal, non-convex). The two best algorithmic variations are when the (GL) selection scheme is combined with the 1-DTC and 1-DBDP partitioning approaches. While the 1-DTDV-GL looks best for more straightforward problems (low-dimensional, uni-modal), the 1-DTC-GL, 1-DBDP-GL combination is more efficient in solving more challenging problems. The worst results were obtained using various partitioning strategies combined with the (IO) selection scheme, which showed promising performance only when a given budget of function evaluations is small.

Moreover, regardless of the selection scheme, the 1-DTDV partitioning strategy has a significant advantage when the most solution coordinates are on the boundary of the feasible region. Additionally, the 1-DTDV partitioning approach has proven to be the most efficient in solving low-dimensional DIRECTGOLib v1.1 test problems. However, the combination based on the 1-DTDV partitioning scheme is very inefficient for higher dimensional

test problems. For such problems, the 1-DBDP partitioning approach seems much more appropriate.

Experimental investigation on 800 GKLS-type test problems showed contrasting results. This study revealed that the (IO) scheme could be very efficient. While on simple GKLS classes, the efficiency of DIRECT-type algorithms based on (IO) and (GL) selection schemes are very similar. However, better performance is explicitly achieved with (IO) for hard classes. Let us recall that this selection scheme showed the worst results in the previous investigation.

To sum up, our study demonstrated that using already known techniques combined in new variations can create more efficient DIRECT-type algorithms. For example, efficient diagonal partition-based BIRECT can be further improved by replacing the original selection scheme with the GL selection (from the DIRECT-GL algorithm), resulting in a more efficient algorithm (1-DBDP-GL).

As for further research, one possible direction could be improving the two-step-based (Global-Local) Pareto selection scheme (GL). Algorithms based on this scheme showed superior performance solving most complex optimization test problems but relatively poor efficiency on more straightforward problems. One possible modification could be borrowing DIRECT's ε parameter or similar technique to limit the size of selected POHs. Optionally, instead of performing the selection enhancing the local search in every iteration, a specific rule could be added to specify when this selection is needed.

Finally, finding the solution efficiently should start by investigating the problem. Then, considering this knowledge, the design or finding of a specific optimization algorithm is needed. Thus, one of our nearest future work plans is to extend this idea by developing the automatic DIRECT-type algorithm selection.

Acknowledgements The authors greatly thank the anonymous Reviewer for his valuable and constructive comments, which helped us significantly extend and improve the manuscript.

Funding The research work of S. Stripinis was funded by a Grant (No. S-MIP-21-53) from the *Research Council of Lithuania*.

Data Availability DIRECTGOLib - DIRECT Global Optimization test problems Library is designed as a continuously-growing open-source GitHub repository to which anyone can easily contribute. The exact data underlying this article from DIRECTGOLib v1.1 can be accessed either on GitHub or at Zenodo:

- at GitHub: <https://github.com/blockchain-group/DIRECTGOLib/tree/v1.1>,
- at Zenodo: <https://doi.org/10.5281/zenodo.6491951>,

and used under the MIT license. We welcome contributions and corrections to it.

Code Availability All twelve introduced DIRECT-type algorithms are implemented in MATLAB and are available in the most recent version of DIRECTGO v1.1.0 (<https://github.com/blockchain-group/DIRECTGO/tree/v1.1.0>) and can be used under the MIT license. We welcome contributions and corrections to it.

A DIRECTGOLib v1.1 library

A summary of all used box-constrained optimization problems from DIRECTGOLib v1.1 [55] and their properties are given in Table 7. Here, the main features are reported: problem number (#), name of the problem, source, dimensionality (n), default optimization domain (D), perturbed optimization domain ($\tilde{D}$), problem type, and the known minimum (f^*). The default domains are taken from the literature. Whenever the global minimum point lies at the initial sampling points for at least one tested algorithm, the domain D has been shifted by 22.5% to the right side. These modified problems are marked with the beta sign ($^\beta$). Here the sign “-” means that $\tilde{D}$ is the same as D . Some of these test problems have several variants, e.g., *Bohachevsky*, *Shekel*, and some of them, like *Alpine*, *Csendes*, *Griewank*, etc., can be tested for varying dimensionality.

Table 7 Key characteristics of the DIRECTGOLib v1.1 [55] test problems for box-constrained global optimization

#	Name	Source	n	D	$\tilde{D}$	Type	No. of minima	f^*
1, 2, 3	Ackley ^{β}	[19, 57]	2, 5, 10	$[-15, 35]^n$	$[-18, 47]^n$	non-convex	multi-modal	0.0000
4, 5, 6	Alpine ^{α}	[12]	2, 5, 10	$[0, 10]^n$	$[\sqrt[3]{2}, 8 + \sqrt[3]{2}]^n$	non-convex	multi-modal	-2.8081^n
7	Beale	[19, 57]	2	$[-4.5, 4.5]^n$	—	non-convex	multi-modal	0.0000
8	Bohachevsky1 ^{β}	[19, 57]	2	$[-100, 110]^n$	$[-55, 145]^n$	convex	uni-modal	0.0000
9	Bohachevsky2 ^{β}	[19, 57]	2	$[-100, 110]^n$	$[-55, 145]^n$	non-convex	multi-modal	0.0000
10	Bohachevsky3 ^{β}	[19, 57]	2	$[-100, 110]^n$	$[-55, 145]^n$	non-convex	multi-modal	0.0000
11	Booth	[19, 57]	2	$[-10, 10]^n$	—	convex	uni-modal	0.0000
12	Branin	[7, 19]	2	$[-5, 10] \times [10, 15]$	—	non-convex	multi-modal	0.3978
13	Bukin6	[57]	2	$[-15, 5] \times [-3, 3]$	—	convex	multi-modal	0.0000
14	Colville	[19, 57]	4	$[-10, 10]^n$	—	non-convex	multi-modal	0.0000
15	Cross_in_Tray	[57]	2	$[0, 10]^n$	—	non-convex	multi-modal	-2.0626
16	Crosslegtable	[12]	2	$[-10, 15]^n$	—	non-convex	multi-modal	-1.000
17, 18, 19	Csendes ^{β}	[12]	2, 5, 10	$[-10, 21]^n$	$[-10, 25]^n$	convex	multi-modal	0.0000
20	Danavandi	[12]	2	$[0, 14]^n$	—	non-convex	multi-modal	0.0000
21, 22, 23	Deb01 ^{β}	[12]	2, 5, 10	$[-1, 1]^n$	$[-0.55, 1.45]^n$	non-convex	multi-modal	-1.0000
24, 25, 26	Deb02 ^{β}	[12]	2, 5, 10	$[0, 1]^n$	$[0.225, 1.225]^n$	non-convex	multi-modal	-1.0000
27, 28, 29	Dixon_and_Price	[19, 57]	2, 5, 10	$[-10, 10]^n$	—	convex	multi-modal	0.0000
30	Drop_wave ^{β}	[57]	2	$[-5.12, 6.12]^n$	$[-4, 6]^n$	non-convex	multi-modal	-1.0000
31	Easom ^{α}	[19, 57]	2	$[-100, 100]^n$	$\left[\frac{-100}{i+1}, 100i \right]^n$	non-convex	multi-modal	-1.0000
32	Eggholder	[57]	2	$[-512, 512]^n$	—	non-convex	multi-modal	-959.6406
33	Goldstein_and_Price ^{β}	[7, 19]	2	$[-2, 2]^n$	$[-1.1, 2.9]^n$	non-convex	multi-modal	3.0000

Table 7 continued

#	Name	Source	n	D	$\tilde{D}$	Type	No. of minima	f^*
34, 35, 36	<i>Griewank^α</i>	[19, 57]	2, 5, 10	$[-600, 700]^n$	$\left[-\sqrt{600i}, \frac{600}{\sqrt{i}}\right]^n$	non-convex	multi-modal	0.0000
37	<i>Hartman3</i>	[19, 57]	3	$[0, 1]^n$	—	non-convex	multi-modal	−3.8627
38	<i>Hartman6</i>	[19, 57]	6	$[0, 1]^n$	—	non-convex	multi-modal	−3.3223
39	<i>Holder_Table</i>	[57]	2	$[-10, 10]^n$	—	non-convex	multi-modal	−19.2085
40	<i>Hump</i>	[19, 57]	2	$[-5, 5]^n$	—	non-convex	multi-modal	−1.0316
41	<i>Langermann</i>	[57]	2	$[0, 10]^n$	—	non-convex	multi-modal	−4.1558
42, 43, 44	<i>Levy</i>	[19, 57]	2, 5, 10	$[-5, 5]^n$	—	non-convex	multi-modal	0.0000
45	<i>Matyas^β</i>	[19, 57]	2	$[-10, 15]^n$	$[-5.5, 14.5]^n$	convex	uni-modal	0.0000
46	<i>McCormick</i>	[57]	2	$[-1.5, 4] \times [-3, 4]$	—	convex	multi-modal	−1.9132
47	<i>Michalewicz</i>	[19, 57]	2	$[0, \pi]^n$	—	non-convex	multi-modal	−1.8013
48	<i>Michalewicz</i>	[19, 57]	5	$[0, \pi]^n$	—	non-convex	multi-modal	−4.6876
49	<i>Michalewicz</i>	[19, 57]	10	$[0, \pi]^n$	—	non-convex	multi-modal	−9.6601
50	<i>Permub4</i>	[19, 57]	4	$[-i, i]^n$	$[-i, i]^n$	non-convex	multi-modal	0.0000
51, 52, 53	<i>Pinter^β</i>	[12]	2, 5, 10	$[-10, 10]^n$	$[-5.5, 14.5]^n$	non-convex	multi-modal	0.0000
54	<i>Powell</i>	[19, 57]	4	$[-4, 5]^n$	—	convex	multi-modal	0.0000
55	<i>Power_Sum^α</i>	[19, 57]	4	$[0, 4]^n$	$[1, 4 + \sqrt[4]{2}]^n$	convex	multi-modal	0.0000
56, 57, 58	<i>Qing</i>	[12]	2, 5, 10	$[-500, 500]^n$	—	non-convex	multi-modal	0.0000
59, 60, 61	<i>Rastrigin^α</i>	[19, 57]	2, 5, 10	$[-6.12, 5.12]^n$	$[-5\sqrt[4]{2}, 7 + \sqrt[4]{2}]^n$	non-convex	multi-modal	0.0000
62, 63, 64	<i>Rosenbrock^α</i>	[7, 19]	2, 5, 10	$[-5, 10]^n$	$\left[-\frac{5}{\sqrt{i}}, 10\sqrt{i}\right]^n$	non-convex	uni-modal	0.0000
65, 66, 67	<i>Rotated_H_Ellip^β</i>	[57]	2, 5, 10	$[-65.536, 66.536]^n$	$[-35, 96]^n$	convex	uni-modal	0.0000

Table 7 continued

#	Name	Source	n	D	$\tilde{D}$	Type	No. of minima	f^*
68, 69, 70	$Schwefel^\alpha$	[19, 57]	2, 5, 10	$[-500, 500]^n$	$\left[-500 + \frac{100}{\sqrt{i}}, 500 - \frac{40}{\sqrt{i}}\right]^n$	non-convex	multi-modal	0.0000
71	$Shekel5$	[19, 57]	4	$[0, 10]^n$	—	non-convex	multi-modal	-10.1531
72	$Shekel7$	[19, 57]	4	$[0, 10]^n$	—	non-convex	multi-modal	-10.4029
73	$Shekel10$	[19, 57]	4	$[0, 10]^n$	—	non-convex	multi-modal	-10.5364
74	$Shubert$	[19, 57]	2	$[-10, 10]^n$	—	non-convex	multi-modal	-186.7309
75, 76, 77	$Sphere^\beta$	[19, 57]	2, 5, 10	$[-5.12, 6.12]^n$	$[-2.75, 7.25]^n$	convex	uni-modal	0.0000
78, 79, 80	$Syblinski_Tang^\alpha$	[4]	2, 5, 10	$[-5, 5]^n$	$[-5, 5 + \sqrt[3]{3}]^n$	non-convex	multi-modal	-39.1661 <i>n</i>
81, 82, 83	$Sum_of_Powers^\beta$	[57]	2, 5, 10	$[-1, 2.5]^n$	$[-0.55, 1.45]^n$	convex	uni-modal	0.0000
84, 85, 86	Sum_Square^β	[4]	2, 5, 10	$[-10, 15]^n$	$[-5.5, 14.5]^n$	convex	uni-modal	0.0000
87	$Trefethen$	[12]	2	$[-2, 2]^n$	—	non-convex	multi-modal	-3.3068
88, 89, 90	$Trid$	[19, 57]	2, 5, 10	$[-100, 100]^n$	—	convex	multi-modal	ϑ
91, 92, 93	$Vincent$	[4]	2, 5, 10	$[0.25, 10]^n$	—	non-convex	multi-modal	- <i>n</i>
94, 95, 96	$Zakharov^\beta$	[19, 57]	2, 5, 10	$[-5, 11]^n$	$[-1.625, 13.375]^n$	convex	multi-modal	0.0000

 $\vartheta = -\frac{1}{6}n^3 - \frac{1}{2}n^2 + \frac{2}{3}n$
 α —domain D was perturbed to avoid the dominance of certain partitioning schemes (see Sect 4.1.2)

 β —domain D was moved by 22.5% percent to the right side

 $i = 1, \dots, n$

References

1. Baker, C.A., Watson, L.T., Grossman, B., Mason, W.H., Haftka, R.T.: Parallel global aircraft configuration design space exploration. In: A. Tentner (ed.) High Performance Computing Symposium 2000, pp. 54–66. Soc. for Computer Simulation Internat (2000)
2. Bartholomew-Biggs, M.C., Parkhurst, S.C., Wilson, S.P.: Using DIRECT to solve an aircraft routing problem. *Comput. Optim. Appl.* **21**(3), 311–323 (2002). <https://doi.org/10.1023/A:1013729320435>
3. Carter, R.G., Gablonsky, J.M., Patrick, A., Kelley, C.T., Eslinger, O.J.: Algorithms for noisy problems in gas transmission pipeline optimization. *Optim. Eng.* **2**(2), 139–157 (2001). <https://doi.org/10.1023/A:1013123110266>
4. Clerc, M.: The swarm and the queen: towards a deterministic and adaptive particle swarm optimization. In: Proceedings of the 1999 Congress on Evolutionary Computation, CEC 1999 (1999). <https://doi.org/10.1109/CEC.1999.785513>
5. Cox, S.E., Haftka, R.T., Baker, C.A., Grossman, B., Mason, W.H., Watson, L.T.: A comparison of global optimization methods for the design of a high-speed civil transport. *J. Glob. Optim.* **21**(4), 415–432 (2001). <https://doi.org/10.1023/A:1012782825166>
6. Di Serafino, D., Liuzzi, G., Piccialli, V., Riccio, F., Toraldo, G.: A modified Dividing RECTangles algorithm for a problem in astrophysics. *J. Optim. Theory Appl.* **151**(1), 175–190 (2011). <https://doi.org/10.1007/s10957-011-9856-9>
7. Dixon, L., Szegö, C.: The global optimisation problem: An introduction. In: L. Dixon, G. Szegö (eds.) *Towards Global Optimization*, vol. 2, pp. 1–15. North-Holland Publishing Company (1978)
8. Finkel, D.E., Kelley, C.T.: An adaptive restart implementation of direct. Technical report CRSC-TR04-30, Center for Research in Scientific Computation, North Carolina State University, Raleigh (2004)
9. Finkel, D.E., Kelley, C.T.: Additive scaling and the DIRECT algorithm. *J. Glob. Optim.* **36**(4), 597–608 (2006). <https://doi.org/10.1007/s10898-006-9029-9>
10. Gablonsky, J.M.: Modifications of the DIRECT algorithm. Ph.D. thesis, North Carolina State University (2001)
11. Gablonsky, J.M., Kelley, C.T.: A locally-biased form of the DIRECT algorithm. *J. Glob. Optim.* **21**(1), 27–37 (2001). <https://doi.org/10.1023/A:1017930332101>
12. Gavana, A.: Global optimization benchmarks and ampgo. http://infinity77.net/global_optimization/index.html. Online; accessed: 2021-07-22
13. Gaviano, M., Kvasov, D.E., Lera, D., Sergeyev, Y.D.: Algorithm 829: Software for generation of classes of test functions with known local and global minima for global optimization. *ACM Trans. Math. Softw. (TOMS)* **29**(4), 469–480 (2003). <https://doi.org/10.1145/962437.962444>
14. Grishagin, V.A.: Operating characteristics of some global search algorithms. In: *Problems of Stochastic Search*, vol. 7, pp. 198–206. Zinatne, Riga. In Russian (1978)
15. He, J., Verstak, A., Sosonkina, M., Watson, L.T.: Performance modeling and analysis of a massively parallel DIRECT-Part 2. *Int. J. High Perform. Comput. Appl.* **23**(1), 29–41 (2009). <https://doi.org/10.1177/1094342008098463>
16. He, J., Verstak, A., Watson, L.T., Sosonkina, M.: Design and implementation of a massively parallel version of direct. *Comput. Optim. Appl.* (2008). <https://doi.org/10.1007/s10589-007-9092-2>
17. He, J., Verstak, A., Watson, L.T., Sosonkina, M.: Performance modeling and analysis of a massively parallel DIRECT-part 1. *Int. J. High Perform. Comput. Appl.* **23**(1), 14–28 (2009). <https://doi.org/10.1177/1094342008098462>
18. He, J., Watson, L.T., Sosonkina, M.: Algorithm 897: VTDIRECT95: serial and parallel codes for the global optimization algorithm DIRECT. *ACM Trans. Math. Softw.* (2010). <https://doi.org/10.1145/1527286.1527291>
19. Hedar, A.: Test functions for unconstrained global optimization. http://www-optima.amp.i.kyoto-u.ac.jp/member/student/hedar/Hedar_files/TestGO.htm (2005). Online; accessed: 2017-03-22
20. Horst, R., Pardalos, P.M., Thoai, N.V.: *Introduction to Global Optimization*. Kluwer Academic Publishers, Nonconvex Optimization and Its Application (1995)
21. Huyer, W., Neumaier, A.: Global optimization by multilevel coordinate search. *J. Glob. Optim.* **14**(4), 331–355 (1999). <https://doi.org/10.1023/A:1008382309369>
22. John, H.: *Adaptation in Natural and Artificial Systems*. The University of Michigan Press, Ann Arbor (1975)
23. Jones, D.R.: The Direct global optimization algorithm. In: Floudas, C.A., Pardalos, P.M. (eds.) *The Encyclopedia of Optimization*, pp. 431–440. Kluwer Academic Publishers, Dordrecht (2001)
24. Jones, D.R., Martins, J.R.R.A.: The DIRECT algorithm: 25 years Later. *J. Glob. Optim.* **79**(3), 521–566 (2021). <https://doi.org/10.1007/s10898-020-00952-6>

25. Jones, D.R., Perttunen, C.D., Stuckman, B.E.: Lipschitzian optimization without the Lipschitz constant. *J. Optim. Theory Appl.* **79**(1), 157–181 (1993). <https://doi.org/10.1007/BF00941892>
26. Kennedy, J., Eberhart, R.: Particle swarm optimization. In *Proceedings of the IEEE International Conference on Neural Networks IV* pp. 1942–1948 (1995). <https://doi.org/10.1109/ICNN.1995.488968>
27. Kirkpatrick, S., Gelatt, C.D., Vecchi, M.P.: Optimization by simulated annealing. *Science* (1983). <https://doi.org/10.1126/science.220.4598.671>
28. Liu, H., Xu, S., Wang, X., Wu, X., Song, Y.: A global optimization algorithm for simulation-based problems via the extended direct scheme. *Eng. Optim.* **47**(11), 1441–1458 (2015). <https://doi.org/10.1080/0305215X.2014.971777>
29. Liu, Q.: Linear scaling and the direct algorithm. *J. Glob. Optim.* **56**, 1233–1245 (2013). <https://doi.org/10.1007/s10898-012-9952-x>
30. Liu, Q., Zeng, J., Yang, G.: MrDIRECT: a multilevel robust DIRECT algorithm for global optimization problems. *J. Glob. Optim.* **62**(2), 205–227 (2015). <https://doi.org/10.1007/s10898-014-0241-8>
31. Liuzzi, G., Lucidi, S., Piccialli, V.: A DIRECT-based approach exploiting local minimizations for the solution of large-scale global optimization problems. *Comput. Optim. Appl.* **45**, 353–375 (2010). <https://doi.org/10.1007/s10589-008-9217-2>
32. Mockus, J., Paulavičius, R., Rusakevičius, D., Šešok, D., Žilinskas, J.: Application of Reduced-set Pareto-Lipschitzian Optimization to truss optimization. *J. Glob. Optim.* **67**(1–2), 425–450 (2017). <https://doi.org/10.1007/s10898-015-0364-6>
33. Paulavičius, R., Chiter, L., Žilinskas, J.: Global optimization based on bisection of rectangles, function values at diagonals, and a set of Lipschitz constants. *J. Glob. Optim.* **71**(1), 5–20 (2018). <https://doi.org/10.1007/s10898-016-0485-6>
34. Paulavičius, R., Sergeyev, Y.D., Kvasov, D.E., Žilinskas, J.: Globally-biased DISIMPL algorithm for expensive global optimization. *J. Glob. Optim.* **59**(2–3), 545–567 (2014). <https://doi.org/10.1007/s10898-014-0180-4>
35. Paulavičius, R., Sergeyev, Y.D., Kvasov, D.E., Žilinskas, J.: Globally-biased BIRECT algorithm with local accelerators for expensive global optimization. *Expert Syst. Appl.* **144**, 11305 (2020). <https://doi.org/10.1016/j.eswa.2019.113052>
36. Paulavičius, R., Žilinskas, J.: Analysis of different norms and corresponding Lipschitz constants for global optimization. *Technol. Econ. Dev. Econ.* **36**(4), 383–387 (2006). <https://doi.org/10.1080/13928619.2006.9637758>
37. Paulavičius, R., Žilinskas, J.: Analysis of different norms and corresponding Lipschitz constants for global optimization in multidimensional case. *Inf. Technol. Control* **36**(4), 383–387 (2007)
38. Paulavičius, R., Žilinskas, J.: Improved Lipschitz bounds with the first norm for function values over multidimensional simplex. *Math. Model. Anal.* **13**(4), 553–563 (2008). <https://doi.org/10.3846/1392-6292.2008.13.553-563>
39. Paulavičius, R., Žilinskas, J.: Global optimization using the branch-and-bound algorithm with a combination of Lipschitz bounds over simplices. *Technol. Econ. Dev. Econ.* **15**(2), 310–325 (2009). <https://doi.org/10.3846/1392-8619.2009.15.310-325>
40. Paulavičius, R., Žilinskas, J.: Simplicial Lipschitz optimization without the Lipschitz constant. *J. Glob. Optim.* **59**(1), 23–40 (2013). <https://doi.org/10.1007/s10898-013-0089-3>
41. Paulavičius, R., Žilinskas, J.: *Simplicial Global Optimization*. SpringerBriefs in Optimization. Springer New York, New York (2014). <https://doi.org/10.1007/978-1-4614-9093-7>
42. Pintér, J.D.: Global optimization in action: continuous and Lipschitz optimization: algorithms, implementations and applications, *Nonconvex Optimization and Its Applications*, vol. 6. Springer US (1996). <https://doi.org/10.1007/978-1-4757-2502-5>
43. Piyavskii, S.A.: An algorithm for finding the absolute minimum of a function. *Theory Optim. Solut.* **2**, 13–24 (1967). [https://doi.org/10.1016/0041-5553\(72\)90115-2](https://doi.org/10.1016/0041-5553(72)90115-2). (In Russian)
44. Rios, L.M., Sahinidis, N.V.: Derivative-free optimization: a review of algorithms and comparison of software implementations. *J. Glob. Optim.* **56**(3), 1247–1293 (2007). <https://doi.org/10.1007/s10898-012-9951-y>
45. Sergeyev, Y.D., Kvasov, D.E.: Global search based on diagonal partitions and a set of Lipschitz constants. *SIAM J. Optim.* **16**(3), 910–937 (2006). <https://doi.org/10.1137/040621132>
46. Sergeyev, Y.D., Kvasov, D.E.: *Diagonal Global Optimization Methods*. FizMatLit, Moscow (2008).. (In Russian)
47. Sergeyev, Y.D., Kvasov, D.E.: Lipschitz global optimization. In: Cochran, J.J., Cox, L.A., Keskinocak, P., Kharoufeh, J.P., Smith, J.C. (eds.) *Wiley Encyclopedia of Operations Research and Management Science* (in 8 volumes), vol. 4, pp. 2812–2828. John Wiley and Sons, New York (2011)
48. Sergeyev, Y.D., Kvasov, D.E.: Deterministic global optimization: an introduction to the diagonal approach. In: *SpringerBriefs in Optimization*. Springer, Berlin (2017). <https://doi.org/10.1007/978-1-4939-7199-2>

49. Shubert, B.O.: A sequential method seeking the global maximum of a function. *SIAM J. Numer. Anal.* **9**, 379–388 (1972). <https://doi.org/10.1137/0709036>
50. Stripinis, L., Paulavičius, R.: DIRECTGO: A new DIRECT-type MATLAB toolbox for derivative-free global optimization (2022). [arxiv:2107.02205](https://arxiv.org/abs/2107.02205)
51. Stripinis, L., Paulavičius, R.: DIRECTGO: A new DIRECT-type MATLAB toolbox for derivative-free global optimization. <https://github.com/blockchain-group/DIRECTGO> (2022)
52. Stripinis, L., Paulavičius, R., Žilinskas, J.: Improved scheme for selection of potentially optimal hyper-rectangles in DIRECT. *Optim. Lett.* **12**(7), 1699–1712 (2018). <https://doi.org/10.1007/s11590-017-1228-4>
53. Stripinis, L., Paulavičius, R., Žilinskas, J.: Penalty functions and two-step selection procedure based DIRECT-type algorithm for constrained global optimization. *Struct. Multidiscip. Optim.* **59**(6), 2155–2175 (2019). <https://doi.org/10.1007/s00158-018-2181-2>
54. Stripinis, L., Paulavičius, R.: DIRECTGOLib - DIRECT Global Optimization test problems Library, v1.0 (2022). <https://doi.org/10.5281/zenodo.6491863>
55. Stripinis, L., Paulavičius, R.: DIRECTGOLib - DIRECT Global Optimization test problems Library, v1.1 (2022). <https://doi.org/10.5281/zenodo.6491951>
56. Strongin, R.G., Sergeyev, Y.D.: *Global Optimization with Non-Convex Constraints: Sequential and Parallel Algorithms*. Kluwer Academic Publishers, Dordrecht (2000)
57. Surjanovic, S., Bingham, D.: Virtual library of simulation experiments: Test functions and datasets. <http://www.sfu.ca/~ssurjano/index.html> (2013). Online; accessed: 2017-03-22
58. Watson, L.T., Baker, C.A.: A fully-distributed parallel global search algorithm. *Eng. Comput.* **18**(1/2), 155–169 (2001). <https://doi.org/10.1108/02644400110365851>
59. Wolpert, D., Macready, W.: No free lunch theorems for optimization. *IEEE Trans. Evol. Comput.* **1**(1), 67–82 (1997). <https://doi.org/10.1109/4235.585893>

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.